FIAP · Global Solution 2026.2 | CitizenTech | Engenharia de Software · 4ES

VigiaVia — Deploy Blindado

MVP de detecção de buracos em vias públicas publicado na nuvem com pipeline DevSecOps, infraestrutura como código e observabilidade

Disciplina **Secure DevOps Tools & Cloud Computing** · Profs. Gabriel Arcanjo e Cleórbete Santos

Integrantes

Nome	RM
Deivison Pertel	550803
Eduardo Akira Murata	98713
Wesley Souza de Oliveira	97874

Links (clicáveis)

MVP publicado (Azure)	https://ca-vigiavia-prod.happywave-0bf5b4c3.brazilsouth.azurecontainerapps.io
Repositório público	https://github.com/akiramrt/vigiavia-mvp
Pipeline DevSecOps	https://github.com/akiramrt/vigiavia-mvp/actions/workflows/devsecops.yml
Execução barrada (Trivy)	https://github.com/akiramrt/vigiavia-mvp/actions/runs/41953686302
Execução corrigida + deploy	https://github.com/akiramrt/vigiavia-mvp/actions/runs/41954209587
Dockerfile · Compose · Terraform	Dockerfile · docker-compose.yml · infra/ · devsecops.yml
Componente de visão computacional	github.com/akiramrt/vigiavia-visao-computacional

Entrega única por equipe · relatório gerado em 05/11/2026 · todas as evidências deste documento são reais, coletadas do ambiente publicado (Azure Brazil South), do GitHub Actions e do Prometheus/Grafana.

1. Resumo e matriz de critérios

O **VigiaVia** detecta buracos em imagens de vias públicas (modelo YOLO26n exportado para ONNX) e entrega à zeladoria um relato único, georreferenciado e priorizado. Nesta disciplina, o MVP foi colocado **no ar no Azure Container Apps**, com toda a infraestrutura em **Terraform**, imagem **Docker** enxuta e sem root, um pipeline **GitHub Actions** que barra segredos, código inseguro e vulnerabilidades antes de qualquer deploy, e **Prometheus + Grafana** medindo os quatro sinais de ouro do serviço publicado.

Critério (peso)	O que foi entregue	Evidência
Container & IaC (5,0)	Dockerfile multi-stage `python:3.12-slim`, usuário 10001, `HEALTHCHECK`, nenhum segredo na imagem; stack completa no `docker compose` (API endurecida + Prometheus + Grafana). Terraform com state remoto no Azure Storage (Entra ID, sem chave), `.tfvars` e `backend.hcl` fora do git, ingress somente HTTPS e **nenhuma porta administrativa**.	Seções 3 e 4; `docs/evidencias/azure_configuracao.txt`; planos em `docs/evidencias/terraform_plan_*.txt`
Pipeline DevSecOps (4,0)	Gitleaks (histórico completo), Semgrep, pytest, validação do compose, Trivy (IaC/Dockerfile e imagem), SBOM; em **todo push**. Credenciais via **OIDC** (sem senha de nuvem) e identificadores em GitHub Secrets. Deploy automático da `main` com smoke test que confere o commit no ar. **Execução barrada e a corrigida.**	Seção 5; runs 41953686302 (barrado) e 41954209587 (corrigido)
Observabilidade (3,0)	API instrumentada (`/metrics` protegido por token); Prometheus coletando o MVP publicado via HTTPS; dashboard Grafana como código com latência, tráfego, erros e saturação; 4 regras de alerta; **alerta disparado** sob carga real; um reinício sob carga foi diagnosticado até a causa raiz, corrigido e verificado em produção.	Seção 6; prints do Grafana e do Prometheus
Documentação (2,0)	Diagrama de arquitetura, responsabilidade compartilhada, onde moram os dados pessoais (LGPD), estimativa de custo com preços oficiais, README reproduzível com a ficha técnica.	Seções 2, 7, 8, 10 e README

Ficha técnica do MVP

Item	VigiaVia
Nome do produto	VigiaVia — zeladoria inteligente de vias públicas
Cidadão e território	Quem circula pelas vias (motociclistas, ciclistas, pedestres, motoristas, usuários de ônibus) e os agentes de zeladoria das subprefeituras. Piloto: centro de São Paulo (Bela Vista).
Problema, em uma frase	Buracos causam acidentes e custos, e sua identificação depende de chamados manuais, lentos, imprecisos e duplicados (113,1 mil pedidos de tapa-buraco em SP em 2025).
Proposta de valor	Detectar buracos automaticamente com as câmeras que já circulam pela cidade e entregar à zeladoria um relato único, georreferenciado e priorizado por buraco — com privacidade e validação humana.
Arquitetura: componentes	API FastAPI + interface web (cidadão e zeladoria) · detector YOLO26n em ONNX Runtime · fila de relatos com deduplicação geográfica · Azure Container Apps + ACR + Log Analytics (Terraform) · pipeline DevSecOps (GitHub Actions) · Prometheus + Grafana.
Fontes de dados	RDD2022 (treino do modelo, CC BY-SA 4.0); relatos e detecções enviados pelos cidadãos; dados de demonstração simulados na Bela Vista (marcados como `demonstracao`).

Modelos de IA utilizados	YOLO26n treinado no RDD2022 (mAP@0,5 = 0,585 no teste), exportado para ONNX; executado no servidor (ONNX Runtime) ou no navegador (onnxruntime-web).
URL do MVP e repositório	https://ca-vigiavia-prod.happywave-0bf5b4c3.brazilsouth.azurecontainerapps.io · https://github.com/akiramrt/vigiavia-mvp
Indicadores de impacto	Tempo entre detecção e reparo · % de chamados duplicados evitados · buracos detectados por km rodado · precisão confirmada pela zeladoria. Operação: disponibilidade, p95 de latência, taxa de erros e saturação.
ODS conectados	ODS 11 (11.2, 11.3) · ODS 9 (9.1) · ODS 3 (3.6)

2. Arquitetura

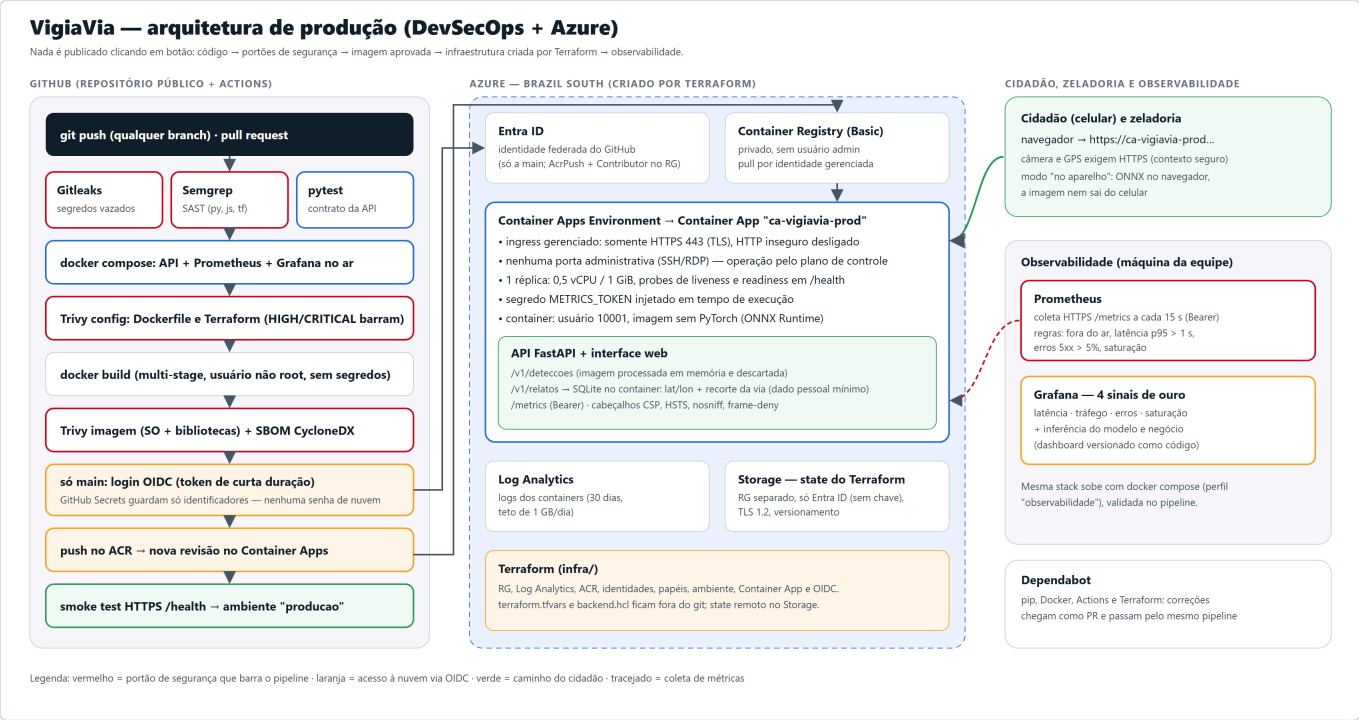


Figura 1 — Arquitetura de produção: o cidadão acessa por HTTPS; o GitHub Actions publica por OIDC; o Prometheus coleta o `/metrics` com token.

Componente	Função e decisão de segurança
Azure Container Apps `ca-vigiavia-prod`	Executa o container da API (0,5 vCPU / 1 GiB, 1 réplica fixa: sem *cold start*). Ingress gerenciado com certificado TLS, **HTTP inseguro desligado**; a porta 8000 do container só é alcançável pelo ingress. Plataforma serverless: **não existe SSH/RDP**.
Azure Container Registry `acrvigiavia2fplg`	Registro privado da imagem aprovada. **Usuário admin desligado**: o app puxa a imagem com identidade gerenciada (AcrPull) e o pipeline publica com identidade federada (AcrPush).
Identidades gerenciadas + Entra ID	`id-vigiavia-prod-app` (pull da imagem) e `id-vigiavia-prod-github` (deploy). A segunda só aceita tokens OIDC da branch `main` deste repositório; papéis restritos ao ACR e ao grupo de recursos.
Log Analytics `log-vigiavia-prod`	Logs da plataforma e do container (stdout/stderr), retenção de 30 dias e teto de 1 GB/dia (controle de custo).
Storage `sttfvigiavia64dba1` (outro grupo)	State remoto do Terraform: autenticação Entra ID, chave compartilhada desligada, TLS 1.2, sem acesso público, versionamento e exclusão reversível por 14 dias.
GitHub Actions	Pipeline DevSecOps: portões de segurança em todo push; publicação e deploy só a partir da `main`.

Prometheus + Grafana	Rodam na máquina da equipe, presos a `127.0.0.1`, coletando o MVP publicado via HTTPS a cada 15 s (Bearer token). O mesmo conjunto sobe com `docker compose --profile observabilidade`.

Tudo fica em **Brazil South** (São Paulo): os dados não saem do país, o que simplifica a LGPD (sem transferência internacional, art. 33) e reduz a latência para o cidadão.

3. Container e stack local

Dockerfile (imagem de produção)

- **Multi-stage**: o estágio `dependencias` instala os pacotes; a imagem final recebe só o resultado, o código, as configurações e o modelo `.onnx`.
- **Base enxuta** `python:3.12-slim-bookworm` com `apt-get upgrade` (correções do SO) e **sem PyTorch**: a inferência usa ONNX Runtime. Imagem de **174 MB** comprimida no ACR.
- **Não root**: usuário `vigia` (UID 10001, sem shell) e `USER 10001`; `/dados` é o único diretório gravável.
- **Sem segredos**: o `METRICS_TOKEN` entra só em tempo de execução (segredo do Container App); o `.dockerignore` exclui `.env`, `*.tfvars`, `infra/`, `.git` e testes.
- `HEALTHCHECK` em `/health`; uvicorn com `--proxy-headers` e `--no-server-header`.
- O commit entra como `ARG COMMIT` e aparece no `/health`: o smoke test prova que a versão nova está no ar.

docker compose (API + Prometheus + Grafana)

- API com sistema de arquivos **somente leitura** (`read_only`, `tmpfs /tmp`), `cap_drop: [ALL]`, `no-new-privileges` e limite de 1 GB de memória.
- `METRICS_TOKEN` obrigatório e lido do `.env` (fora do git); senha do Grafana idem.
- Prometheus e Grafana **presos a 127.0.0.1**: as interfaces administrativas nunca ficam expostas na rede.
- Dashboard e alertas versionados como código (`observabilidade/`), provisionados automaticamente.
- O job **"Stack docker compose"** do pipeline sobe a stack completa em todo push e só passa se o Prometheus estiver coletando a API (`up == 1`).

Na própria aplicação: limite de 8 MB e 40 MP por imagem, a imagem enviada para detecção **é descartada** após a resposta, e todas as respostas levam cabeçalhos de segurança (CSP restritiva, HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Permissions-Policy`). O `/metrics` exige *Bearer token* comparado em tempo constante (`hmac.compare_digest`).

4. Infraestrutura como código (Terraform)

Requisito	Como foi atendido
Tudo em código	`infra/main.tf` cria grupo de recursos, Log Analytics, ACR, identidades gerenciadas, RBAC, ambiente e app do Container Apps e a credencial federada do GitHub. Provedores fixados (`azurerm 4.81.0`, `random 3.9.1`). O drift detectado (perfil de carga "Consumption" que o Azure adiciona) foi declarado no código.
State fora do repositório	Backend `azurerm` no Storage `sttfvigiavia64dba1`, criado por `infra/bootstrap/criar_backend.sh`: acesso por Entra ID (`use_azuread_auth`), **chave compartilhada desligada**, versionamento e retenção de exclusão. O `backend.hcl` é gerado localmente e ignorado pelo git.
`.tfvars` fora do repositório	`terraform.tfvars` (assinatura e `metrics_token`) é gerado por `scripts/preparar_segredos.py` e está no `.gitignore` (`*.tfvars`, `*.tfstate*`, `*.tfplan`, `backend.hcl`, `.env`). A variável do token é `sensitive`.
Porta admin nunca aberta a 0.0.0.0/0	Não há VM nem porta administrativa: o único ponto de entrada é o ingress HTTPS (`allow_insecure_connections = false`, sem portas adicionais). A operação acontece pelo plano de controle do Azure, autenticado e auditado. O Trivy (config) e o Semgrep (`p/terraform`) revisam o Terraform em todo push.

Menor privilégio	ACR sem admin; app com **AcrPull**; identidade do GitHub com **AcrPush** só no ACR e **Contributor** só no grupo `rg-vigiavia-prod`; credencial federada com *subject* que usa os IDs imutáveis do dono e do repositório (`repo:akiramrt@128194173/vigiavia-mvp@1446365122:ref:refs/heads/main`).
Revisão humana do apply	Cada mudança gera um `terraform plan -out` salvo como evidência (`docs/evidencias/terraform_plan_*.txt`, sem valores sensíveis); o `apply` do plano revisado é executado por um integrante.

```
## Ingress do Container App: só HTTPS, HTTP inseguro desligado, nenhuma porta administrativa
{
  "externo": true,
  "fqdn": "ca-vigiavia-prod.happywave-0bf5b4c3.brazilsouth.azurecontainerapps.io",
  "httpInseguroPermitido": false,
  "portaDoContainer": 8000,
  "portasAdicionais": null,
  "transporte": "Http"
}

## Probes do container (após o ajuste de 05/11: timeout de 5 s)
Tipo       Caminho    IntervaloSeg    TimeoutSeg    FalhasParaAgir
---------  ---------  --------------  ------------  ----------------
Liveness   /health    30              5             3
Readiness  /health    10              5             3

## Registro de imagens: usuário admin desligado (acesso só por identidade + RBAC)
{
  "acessoPublico": "Enabled",
  "adminHabilitado": false,
  "registro": "acrvigiavia2fplg.azurecr.io",
  "sku": "Basic"
}

## Identidade federada do GitHub (OIDC): só a branch main deste repositório
[
  {
    "audiencia": "api://AzureADTokenExchange",
    "emissor": "https://token.actions.githubusercontent.com",
    "nome": "github-main",
    "subject": "repo:akiramrt@128194173/vigiavia-mvp@1446365122:ref:refs/heads/main"
  }
]

## Papéis da identidade do GitHub (menor privilégio)
Papel        Escopo
-----------  ------------------------------------------------------------------------------------------------------------------------
-------------------------------------
Contributor  /subscriptions/…/resourceGroups/rg-vigiavia-prod
AcrPush      /subscriptions/…/resourceGroups/rg-vigiavia-
prod/providers/Microsoft.ContainerRegistry/registries/acrvigiavia2fplg

## State remoto do Terraform (Storage com Entra ID, sem chave compartilhada, versionado)
{
  "blobPublico": false,
  "chaveCompartilhada": false,
  "conta": "sttfvigiavia64dba1",
  "somenteHttps": true,
  "tlsMinimo": "TLS1_2"
}
{
  "retencaoExclusaoDias": 14,
  "versionamento": true
}
```

Trecho de `docs/evidencias/azure_configuracao.txt` (Azure CLI): ingress só HTTPS e sem portas adicionais, ACR sem admin, OIDC restrito à `main`, papéis mínimos e state protegido.

5. Pipeline DevSecOps (GitHub Actions)

O workflow `devsecops.yml` roda em **todo push** (qualquer branch), em pull requests e sob demanda. Permissões mínimas por padrão (`contents: read`); `id-token: write` só nos jobs que falam com o Azure. As actions de terceiros são

fixadas pelo **SHA do commit** (tags podem ser movidas) e o Dependabot acompanha pip, Docker, Actions e Terraform.

Job / portão	O que barra
Segredos (Gitleaks)	Segredos em **todo o histórico** (`fetch-depth: 0`) — apagar a linha num commit novo não basta. Binário com versão e SHA-256 conferidos.
SAST (Semgrep)	Código inseguro em Python e JavaScript, segredos, Dockerfile e Terraform (`p/python p/javascript p/secrets p/dockerfile p/terraform` , `--error`).
Testes (pytest)	Regressões da API: detecção real com o modelo ONNX, deduplicação, validação humana, proteção do `/metrics` , cabeçalhos.
Stack docker compose	Stack que não sobe ou Prometheus que não coleta a API.
Build + Trivy	Configuração insegura (Dockerfile/Terraform) e vulnerabilidades **HIGH/CRITICAL corrigíveis** no SO e nas bibliotecas da imagem; gera o SBOM CycloneDX (artefato do run).
Push no ACR	Só a imagem aprovada, só na `main` , com login OIDC.
Deploy + smoke test	`az containerapp update` com a tag do commit; o smoke test só passa quando o `/health` público responde com **o commit implantado**.

> **Segredos.** Não existe senha de nuvem no GitHub: o job pede ao GitHub um token OIDC de curta duração e o Entra ID o troca por acesso, porque confia na credencial federada criada pelo Terraform. Os GitHub Secrets guardam só identificadores (`AZURE_CLIENT_ID` , `AZURE_TENANT_ID` , `AZURE_SUBSCRIPTION_ID` , `ACR_NOME` , `ACA_NOME` , `AZURE_RG`); as Variables `PUBLICAR_IMAGEM` e `DEPLOY_HABILITADO` ligam a publicação. O único segredo de verdade, o `METRICS_TOKEN` , vive no `.tfvars` local e no cofre de segredos do Container App.

Execução barrada e a corrigida

Demonstração controlada. O commit `709a483` fixou o `python-multipart` (o parser dos uploads de imagem do cidadão) na versão antiga 0.0.17. Os testes passaram (a funcionalidade não muda), mas o Trivy encontrou **4 vulnerabilidades HIGH** na imagem, entre elas a CVE-2024-53981 (negação de serviço pelo *boundary* do multipart) e a CVE-2026-24486 (escrita arbitrária de arquivo por *path traversal*). O run 41953686302 parou no job "Build + Trivy": a imagem não foi publicada, o deploy foi pulado e a produção continuou no commit anterior (`cd0c523` , conferido no `/health`).

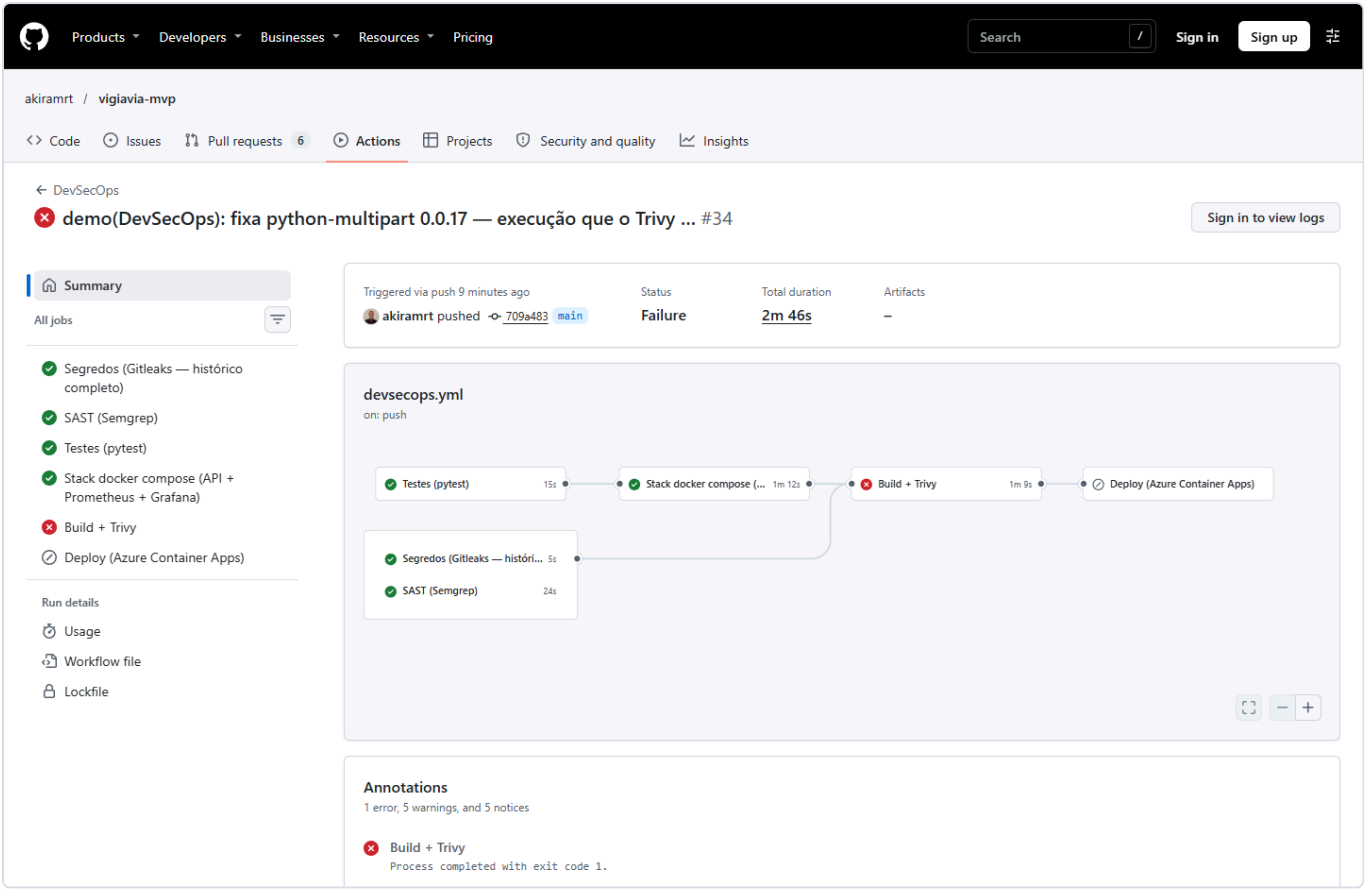


Figura 2 — Run 41953686302: Gitleaks, Semgrep, pytest e compose passaram; o Trivy barrou a imagem e o deploy foi pulado.

Biblioteca	CVE	Severidade	Instalada	Corrigida em	Descrição (Trivy)
python-multipart	CVE-2024-53981	**HIGH**	0.0.17	0.0.18	python-multipart has a DoS via deformation `multipart/form-data` boundary
python-multipart	CVE-2026-24486	**HIGH**	0.0.17	0.0.22	Arbitrary file write via path traversal vulnerability
python-multipart	CVE-2026-42561	**HIGH**	0.0.17	0.0.27	Denial of Service via excessive multipart part headers
python-multipart	CVE-2026-53539	**HIGH**	0.0.17	0.0.30	Denial of Service via crafted form-urlencoded bodies

```
Python (python-pkg)
Total: 4 (HIGH: 4, CRITICAL: 0)
##[error]Process completed with exit code 1.   → push no ACR e deploy NÃO executados
```

A correção (`91f8225`) voltou para a versão 0.0.33. O run 41954209587 passou por todos os portões, publicou a imagem no ACR e fez o deploy automático; o smoke test confirmou o commit novo no ar.

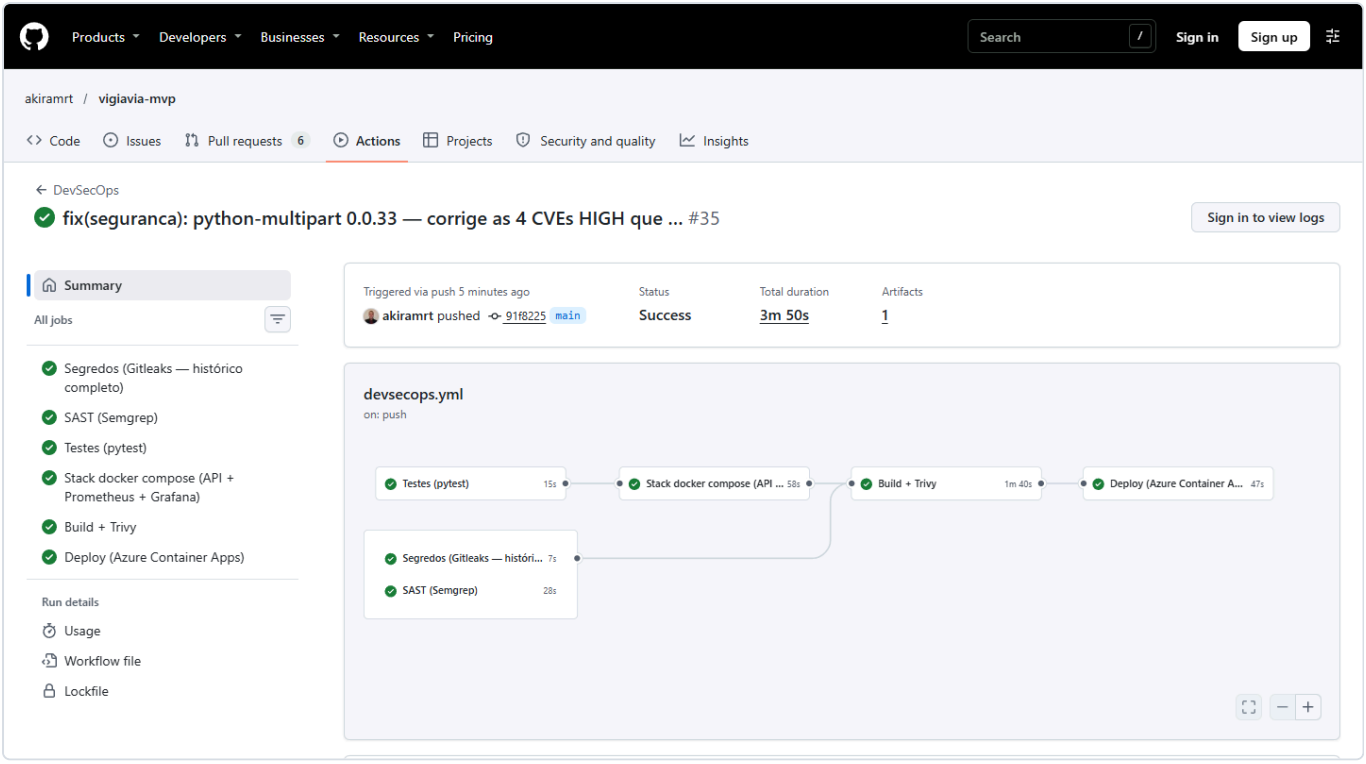


Figura 3 — Run 41954209587: todos os portões verdes, deploy no Azure Container Apps e smoke test.

```
Revisao  Imagem  Saude  Estado
-----------------------  ------------------------------------------------------------------------------------  -------  -----
---
***--latest  ***.azurecr.io/vigiavia-mvp:91f82257712cb1de99f06fb4bc9dcc6675a955c6  None  Running
{"status":"ok","versao":"1.1.0","commit":"91f82257712cb1de99f06fb4bc9dcc6675a955c6","modelo":"vigiavia_buracos","limiar_con
fianca":0.305}
MVP no ar com o commit 91f82257712cb1de99f06fb4bc9dcc6675a955c6: https://***.happywave-
0bf5b4c3.brazilsouth.azurecontainerapps.io
```

Log do job de deploy: os `***` são valores de GitHub Secrets (nome do app e do registro) mascarados automaticamente pelo Actions.

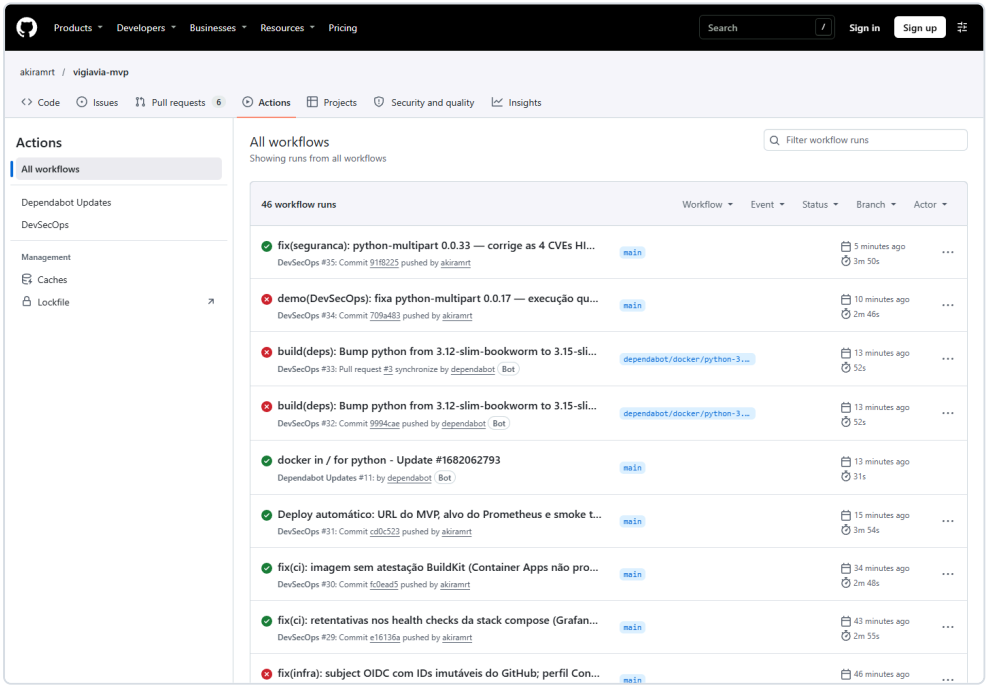


Figura 4 — Histórico do pipeline: no topo, a sequência deploy → barrado → corrigido; no meio, PRs do Dependabot que propunham Python 3.15 e foram barrados; abaixo, os ajustes iniciais do próprio pipeline (seção 9).

6. Observabilidade — os quatro sinais de ouro

A API exporta métricas Prometheus próprias (`http_requests_total{method,rota,status}`, histograma `http_request_duration_seconds`, *gauge* `http_requests_em_andamento`, `vigiavia_inferencia_segundos`, `vigiavia_buracos_detectados_total`, `vigiavia_relatos_total`) além das métricas de processo (CPU e memória). O rótulo `rota` usa o **modelo da rota** (ex.: `/v1/relatos/{rid}`), nunca o caminho bruto: sem explosão de cardinalidade e sem dado pessoal em métrica.

Sinal	Consulta (PromQL) no dashboard	Alerta
Latência	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{rota="/v1/deteccoes"}[5m])))` + p50/p95/p99 por minuto	`VigiaViaLatenciaAlta`: p95 da detecção > 1 s por 1 min
Tráfego	`sum by (rota) (rate(http_requests_total[1m]))`	—
Erros	`sum(rate(http_requests_total{status=~"5.."}[1m])) / sum(rate(http_requests_total[1m]))` (e 4xx)	`VigiaViaErros5xx`: > 5% por 2 min
Saturação	`http_requests_em_andamento`, `rate(process_cpu_seconds_total[1m])` (limite: 0,5 vCPU) e memória residente (limite: 1 GiB)	`VigiaViaSaturada`: ≥ 6 requisições em andamento por 1 min
Disponibilidade	`up` do alvo publicado	`VigiaViaForaDoAr`: `up == 0` por 1 min

Como foi medido. O Prometheus da equipe (127.0.0.1) coleta `https://…azurecontainerapps.io/metrics` a cada 15 s com o token em arquivo (`credentials_file`). Para provocar os sinais, `scripts/gerar_carga.py` disparou 16 clientes simultâneos por 7 minutos (85% `POST /v1/deteccoes` com imagens reais de amostra e 15% `GET /v1/relatos`) contra o MVP publicado. Resultado do gerador: **3.498 requisições em 420 s (8,3 req/s), 99,9% respondidas com sucesso, latência p50 de 1,93 s e p95 de 2,66 s**. O primeiro alerta disparou aos 114 s de carga. No momento dos prints, o Prometheus mostrava em **FIRING**: `VigiaViaLatenciaAlta` (valor 1,92 s).

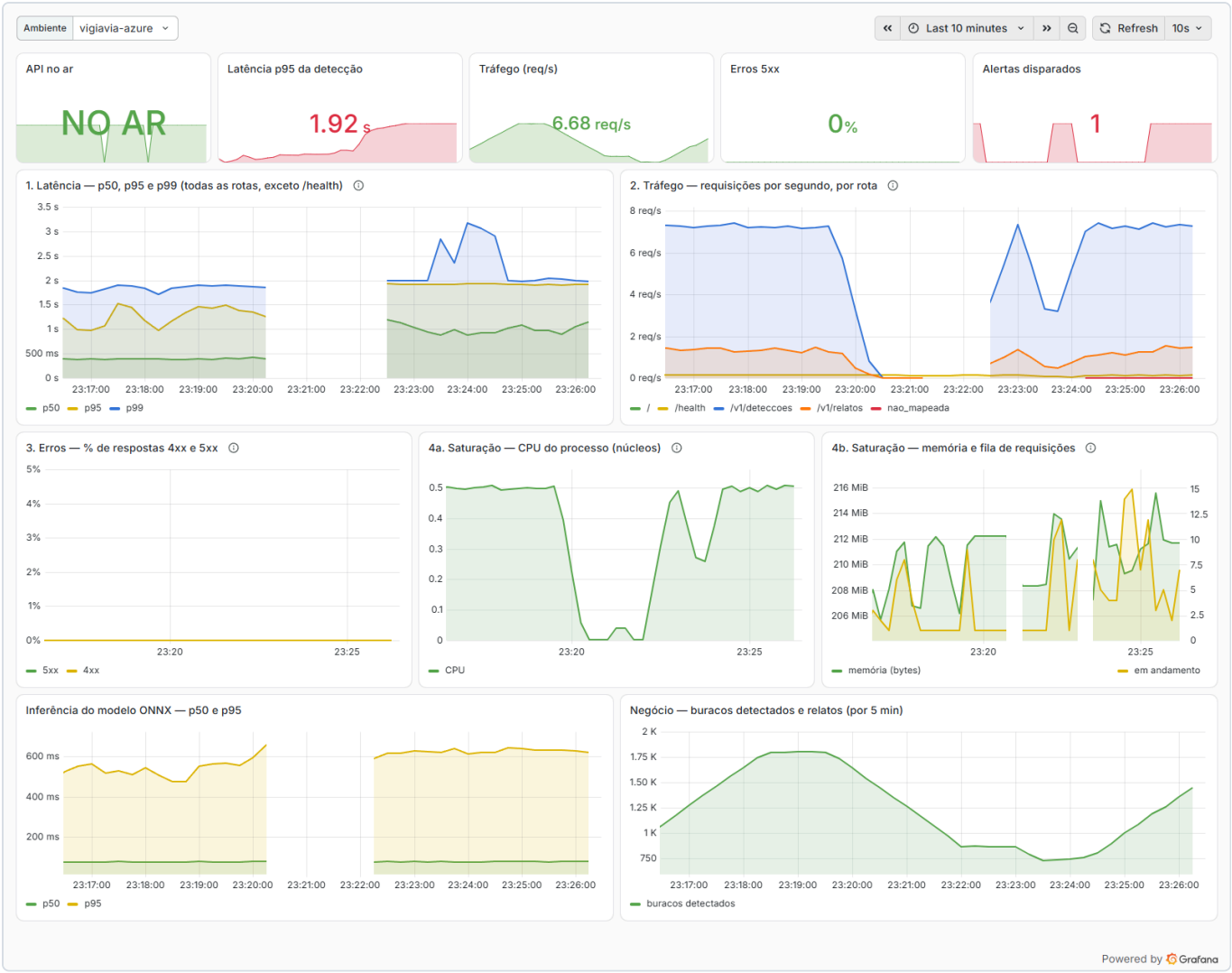


Figura 5 — Dashboard "VigiaVia — sinais de ouro" (Grafana) com duas rodadas de carga contra o MVP publicado: 10 clientes (até 23:19:30) e 16 clientes (a partir de 23:21:40). O intervalo sem dados entre elas inclui a troca de revisão do deploy automático.

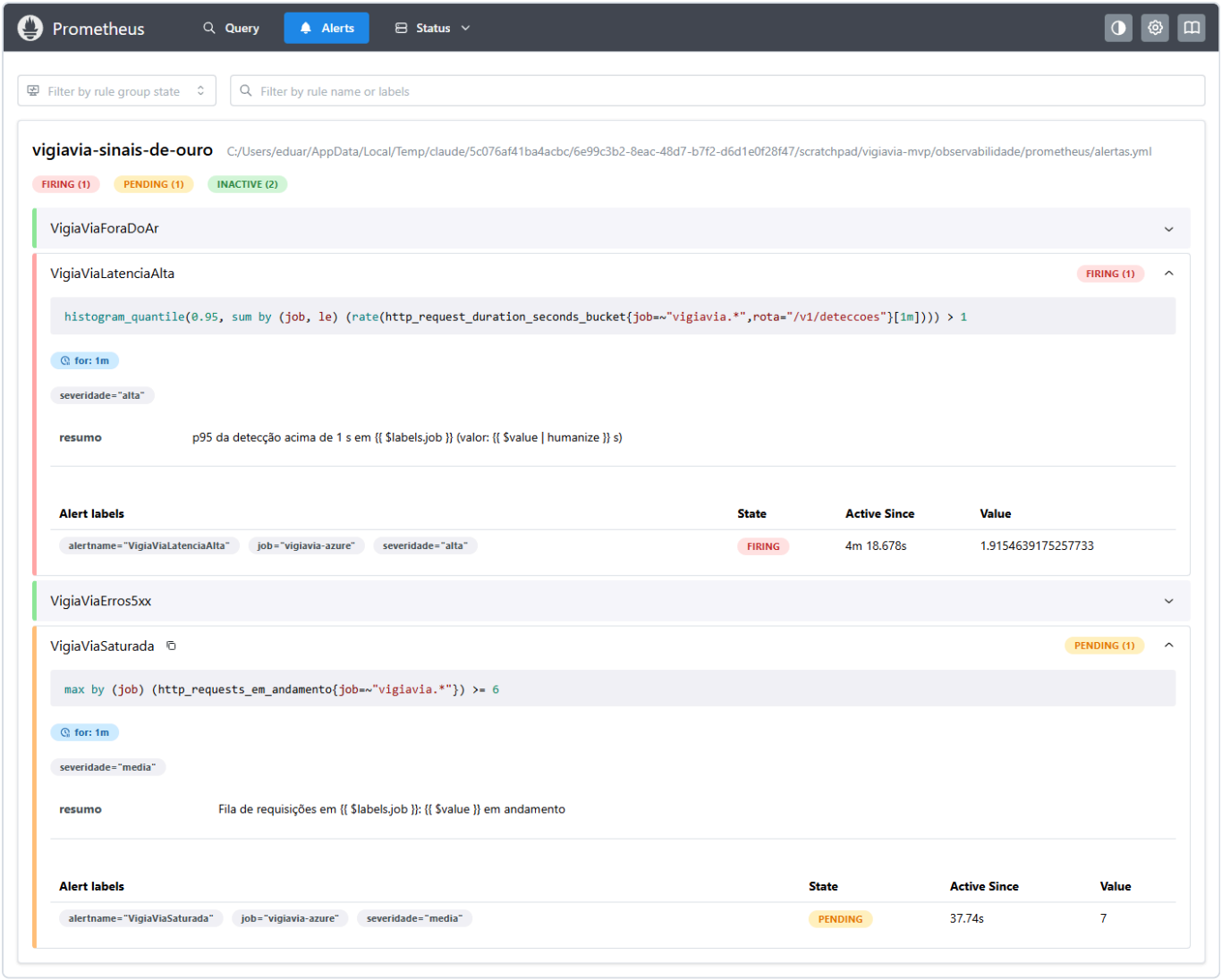


Figura 6 — Prometheus durante a carga: `VigiaViaLatenciaAlta` em FIRING (p95 = 1,92 s) e `VigiaViaSaturada` em PENDING (7 requisições em andamento).

Leitura. Com 1 réplica de 0,5 vCPU a detecção satura em cerca de 8,3 req/s: a CPU encosta no limite, a fila chega a 15 requisições em andamento, o p95 passa de 1 s e o alerta de latência dispara — sem nenhum 5xx registrado pela API. O dashboard também revelou um efeito colateral real: às 23:23, cerca de 90 s após o início da rodada de 16 clientes, o processo reiniciou (métrica `process_start_time_seconds`, queda no tráfego) — 0,1% das requisições do gerador falharam.

Do alerta à causa raiz.

1. **Hipótese 1 — timeout da *liveness probe*** (1 s, padrão). Subimos para 5 s nas duas probes pelo Terraform (plano revisado em `docs/evidencias/terraform_plan_6_probes.txt` e aplicado por um integrante). **Reteste com 16 clientes: o container ainda reiniciou 2 vezes** (`restartCount = 2` no Azure). Hipótese refutada.
2. **Causa raiz.** A rota `POST /v1/deteccoes` é `async` e executava a decodificação da imagem e a inferência ONNX **dentro do event loop**: com a fila de detecções, o `/health` consultado pela probe esperava atrás de todas elas. Um teste de regressão reproduziu o defeito — o `/health` esperou 1,22 s atrás de uma única inferência lenta.
3. **Correção.** O trabalho de CPU passou para threads de trabalho (`anyio.to_thread`), com no máximo 2 inferências simultâneas (`VIGIAVIA_INFERENCIAS_SIMULTANEAS`); o teste de regressão agora passa no pipeline. O timeout de 5 s ficou como defesa em profundidade.
4. **Verificação em produção** (16 clientes por 5 min, commit `b12e62e`): 2.482 requisições (8,3 req/s), **100,0% com sucesso**, p95 de 2,82 s (a CPU continua no limite: escalar ainda é necessário), fila de até 17 requisições, `/health` **sempre 200** com latência mediana de 0,71 s e máxima de 3,74 s nas 24 medições, e **nenhum reinício** (`restartCount = 0`; `process_start_time_seconds` constante). Amostras em `docs/evidencias/reteste_carga_pos_correcao.txt`.

Próximo passo: escala horizontal por requisições concorrentes (regra HTTP do Container Apps), o cenário "Cidade" da seção 8.

Observação: nesta máquina o Docker Desktop não iniciou (WSL desativado), então Prometheus e Grafana rodaram como binários nativos com os **mesmos arquivos de configuração** do repositório, presos a 127.0.0.1; a stack em contêineres é validada pelo job de compose a cada push.

7. Segurança, responsabilidade compartilhada e LGPD

Modelo de responsabilidade compartilhada (PaaS / contêiner serverless)

Microsoft (provedor)	Compartilhado	Equipe VigiaVia (cliente)
Data centers, hardware, rede física, hipervisor e sistema dos hosts; plano de controle e orquestração do Container Apps; disponibilidade e correções da plataforma; certificado e terminação TLS do ingress; criptografia em repouso padrão do Storage e do ACR; serviço do Entra ID.	Identidade (o Entra ID fornece; nós configuramos identidades, RBAC e a federação OIDC); rede (a plataforma isola; nós decidimos o que é exposto — só HTTPS); logs (a plataforma coleta; nós definimos retenção, acesso e custo).	Código e dependências; imagem base e suas correções (rebuild + Trivy); configuração do container (não root, probes, limites); segredos (`METRICS_TOKEN`, state); IaC e proteção do state; cadeia de CI/CD (actions fixadas, OIDC); dados dos cidadãos (classificação, minimização, retenção, LGPD); monitoramento e resposta a alertas.

Onde moram os dados pessoais (LGPD)

Dado	Onde fica	Tratamento e retenção
Imagem enviada para detecção (pode conter rostos e placas)	Só na memória do container, durante a requisição	**Não é armazenada**: descartada após a resposta. Também é possível detectar **no próprio navegador** (onnxruntime-web), sem enviar a imagem.
Relato: latitude/longitude, severidade, data/hora e recorte da via	SQLite em `/dados` no container do Azure Container Apps (Brazil South)	Minimização: só o recorte do buraco (JPEG ≤ 200 KB validado), sem identificação do cidadão. A localização pode revelar onde alguém estava — por isso não há login, nome nem identificador de aparelho. No MVP o disco é efêmero (dados recriados a cada revisão); em produção: banco gerenciado em Brazil South com criptografia, backup e política de retenção (ex.: 12 meses após o reparo).
Endereço IP do cliente nos logs de acesso	Log Analytics `log-vigiavia-prod` (Brazil South)	Dado pessoal para fins de segurança; retenção de **30 dias**, acesso restrito por RBAC, teto de 1 GB/dia.
Métricas	Prometheus da equipe	Sem dado pessoal: rótulos só de método, modelo de rota e status.

Segredos e state do Terraform	Storage privado (Entra ID) e cofre do Container App	Não são dados pessoais, mas o state contém o token: nunca vai para o git; versionado e com acesso só por identidade.

Base legal: execução de políticas públicas pela administração (LGPD, art. 7º, III, e art. 23), com transparência ao cidadão (finalidade: zeladoria de vias). A validação humana na fila da zeladoria evita decisões automatizadas sobre o cidadão.

8. Estimativa de custo

Calculada por `scripts/estimar_custo.py` com a **Azure Retail Prices API** (Brazil South, preços de varejo em USD) e a **PTAX do Banco Central** de 04/11/2026 (R$ 5,4778). Resultado em `docs/custos.json`.

Item (Brazil South)	MVP (1 réplica 0,5 vCPU / 1 GiB, 5% ativa)	Cidade (4 réplicas 1 vCPU / 2 GiB, 40% ativas)
Container Apps — vCPU ativa	US$ 0,00	US$ 96,60
Container Apps — vCPU ociosa	US$ 3,74	US$ 18,92
Container Apps — memória	US$ 6,80	US$ 61,99
Container Apps — requisições	US$ 0,00	US$ 7,20
Container Registry Basic	US$ 5,06	US$ 5,06
Log Analytics (ingestão)	US$ 0,00	US$ 0,00
Storage do state (Terraform)	US$ 0,00	US$ 0,00
Total mensal	**US$ 15,61 · R$ 85,53**	**US$ 189,77 · R$ 1.039,54**
Requisições/mês consideradas	300.000	20.000.000

O MVP custa cerca de **US$ 15,61/mês (R$ 85,53)** e cabe no crédito do Azure for Students. Os maiores itens são a réplica sempre ligada (sem *cold start*) e o ACR Basic. As franquias mensais gratuitas do Container Apps (vCPU-s, GiB-s e 2 milhões de requisições) e os 5 GB de ingestão do Log Analytics zeram os demais itens. Na escala de cidade (4 réplicas de 1 vCPU / 2 GiB, 20 milhões de requisições/mês) o custo vai a **US$ 189,77/mês (R$ 1.039,54)**.

9. Problemas reais encontrados e como foram resolvidos

Problema	Causa e solução
Login OIDC falhava (`AADSTS700213`)	O GitHub passou a emitir o claim `sub` com IDs imutáveis (`repo:dono@id/repo@id:ref:…`). O *subject* virou variável do Terraform com esse formato — o que ainda impede que um repositório recriado com o mesmo nome herde o acesso.
Container App falhava: "Artifact app provisioning failed"	O BuildKit cria por padrão um índice OCI com atestação de plataforma `unknown/unknown` , que o Container Apps não aceita. Build com `BUILDX_NO_DEFAULT_ATTESTATIONS=1` ; a rastreabilidade continua no SBOM CycloneDX.
Deploy com `environment:` quebraria o OIDC	Com um *environment* o `sub` vira `…:environment:<nome>` . O job de deploy roda sem ele, e a credencial federada continua confiando só na `main` .
Action do Trivy "não encontrada"	A tag usada deixou de existir (as tags passaram a ter prefixo `v`). Todas as actions foram fixadas por SHA — tags são mutáveis e já foram vetor de ataque à cadeia de suprimentos.
Gitleaks falhava no primeiro push	A action varria um intervalo `commit^..` inexistente no commit raiz. Troca pelo binário (versão e SHA-256 fixados) varrendo o histórico completo.
Drift no ambiente do Container Apps	O Azure adiciona o perfil "Consumption" e o plano queria recriar o ambiente. O perfil foi declarado no Terraform (o código continua sendo a fonte da verdade).
Job de compose intermitente	Prometheus e Grafana não têm `HEALTHCHECK` ; o teste batia antes de aceitarem conexões. `curl --retry --retry-all-errors` .

Container reiniciava sob carga	Visto no dashboard durante o teste de carga. Aumentar o timeout das probes (1 s → 5 s, via Terraform) não resolveu; a causa era a inferência rodando no event loop da rota `async`, que travava o `/health`. Inferência em threads com limite de 2 simultâneas, teste de regressão e reteste em produção sem reinício (seção 6).

10. Como reproduzir

```
# 1) Local (Docker): a mesma stack do pipeline
python scripts/preparar_segredos.py             # .env e token do Prometheus (fora do git)
docker compose up -d --build                    # API em http://localhost:8000
docker compose --profile observabilidade up -d  # + Prometheus :9090 e Grafana :3000 (só 127.0.0.1)

# 2) Portões de segurança na máquina (os mesmos do pipeline)
pip install -r requirements-dev.txt && pytest -q
gitleaks git . && trivy config . && semgrep scan --config p/python --config p/secrets --error

# 3) Nuvem, uma vez: state remoto, variáveis fora do git, plano revisado por uma pessoa
az login && bash infra/bootstrap/criar_backend.sh brazilsouth
python scripts/preparar_segredos.py --subscription <assinatura>   # infra/terraform.tfvars
cd infra && terraform init -backend-config=backend.hcl && terraform plan -out=plano.tfplan
terraform apply plano.tfplan
powershell -File ..\scripts\configurar_github.ps1 -HabilitarDeploy   # GitHub Secrets/Variables (OIDC)

# 4) Daí em diante: git push na main = portões de segurança + deploy automático + smoke test
```

Passo a passo completo, com a ficha técnica, no README. Telas do MVP publicado:

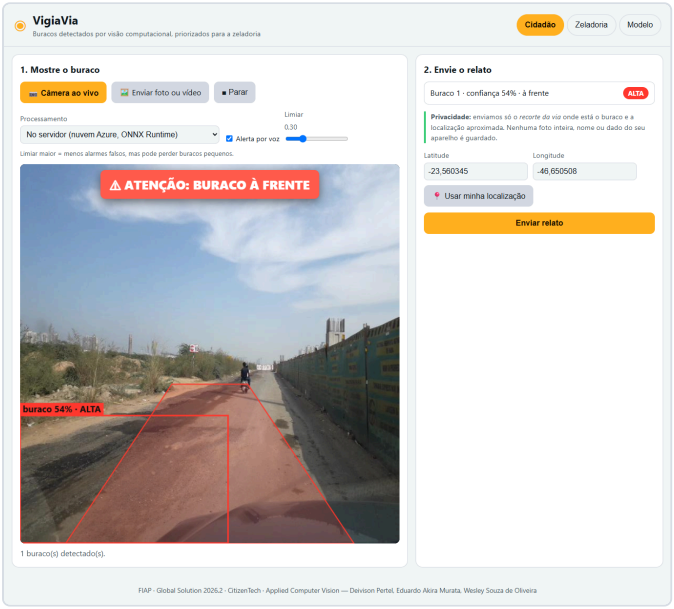


Figura 7 — Visão do cidadão: detecção de buraco numa imagem de exemplo.

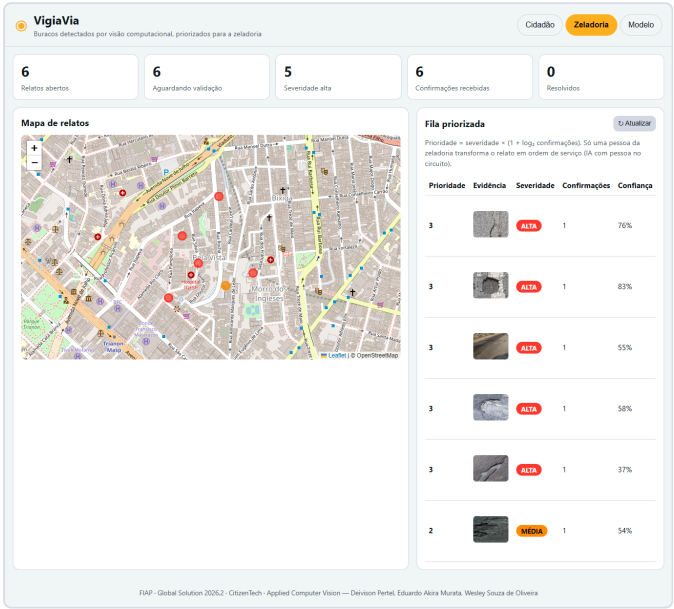


Figura 8 — Visão da zeladoria: fila priorizada e mapa dos relatos.

11. Conclusão

O VigiaVia está no ar em https://ca-vigiavia-prod.happywave-0bf5b4c3.brazilsouth.azurecontainerapps.io e só chega lá passando por portões automáticos: nenhum segredo no histórico, nenhum padrão inseguro no código, testes e stack funcionando, nenhuma vulnerabilidade HIGH/CRITICAL corrigível na imagem. A demonstração mostrou o ponto central do DevSecOps: uma mudança que **passa nos testes** pode ser **perigosa**, e foi o portão de segurança — não uma revisão manual — que impediu a imagem vulnerável de chegar ao cidadão. A infraestrutura é reproduzível por código, sem senhas de nuvem e sem portas administrativas, e a observabilidade mostrou, com carga real, onde o serviço satura e quando precisa escalar — e revelou um defeito (a inferência travando o event loop e derrubando a liveness probe) que foi investigado até a causa raiz, corrigido com teste de regressão e verificado em produção.